

Отчёт по лабораторной работе №7

Дисциплина: Архитектура компьютеров

Мухина Ксения Николаевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Задания для самостоятельной работы	12
4	Выводы	18
	Список литературы	19

Список иллюстраций

2.1	Создание каталога и файла для работы.	6
2.2	Программа lab7-1.	6
2.3	Изменение программы lab7-1.	7
2.4	Изменённая программа lab7-1.	7
2.5	Изменение программы lab7-1.	8
2.6	Изменённая программа lab7-1.	8
2.7	Программа lab7-2.	9
2.8	Создание файла листинга lab7-2.lst.	9
2.9	Файл листинга lab7-2.lst.	10
2.10	Изменение программы lab7-2.	11
2.11	Изменённый фрагмент листинга lab7-2.	11
3.1	Программа task1.asm.	14
3.2	Программа task2.asm.	17

Список таблиц

1 Цель работы

Изучить команды условного и безусловного переходов в NASM. Приобрести навыки написания программ с использованием переходов. Ознакомиться с назначением и структурой файла листинга.

2 Выполнение лабораторной работы

Далее описываемая работа была выполнена на виртуальной машине Oracle VirtualBox с ОС Linux Ubuntu.

1. Реализация переходов в NASM

Перед началом работы создадим каталог lab07 и файл lab7-1.asm.

```
knmuhina@virtualbox:~$ mkdir ~/work/arch-pc/lab07
knmuhina@virtualbox:~$ cd ~/work/arch-pc/lab07
knmuhina@virtualbox:~/work/arch-pc/lab07$ touch lab7-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab07$
```

Рисунок 2.1: Создание каталога и файла для работы.

Рассмотрим первый пример использования инструкции jmp. Введём в файл текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл.

```
knmuhina@virtualbox:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
knmuhina@virtualbox:~/work/arch-pc/lab07$
```

Рисунок 2.2: Программа lab7-1.

Инструкция jmp _label2 меняет порядок исполнения инструкции и позволяет пропустить вывод первого сообщения, начиная выполнение инструкций с метки _label2.

Инструкция `jmp` позволяет осуществлять переходы не только вперёд, но и назад. Изменим текст программы следующим образом:

```
_start:                                _start:
jmp _label2                            jmp _label2

_label1:                               _label1:
mov eax, msg1                          mov eax, msg1
call sprintfLF                         call sprintfLF
jmp _end                               jmp _end

_label2:                               _label2:
mov eax, msg2                          mov eax, msg2
call sprintfLF                         call sprintfLF
jmp _label1                            jmp _label1

_label3:                               _label3:
mov eax, msg3                          mov eax, msg3
call sprintfLF                         call sprintfLF

_end:                                  _end:
call quit                              call quit
```

Рисунок 2.3: Изменение программы lab7-1.

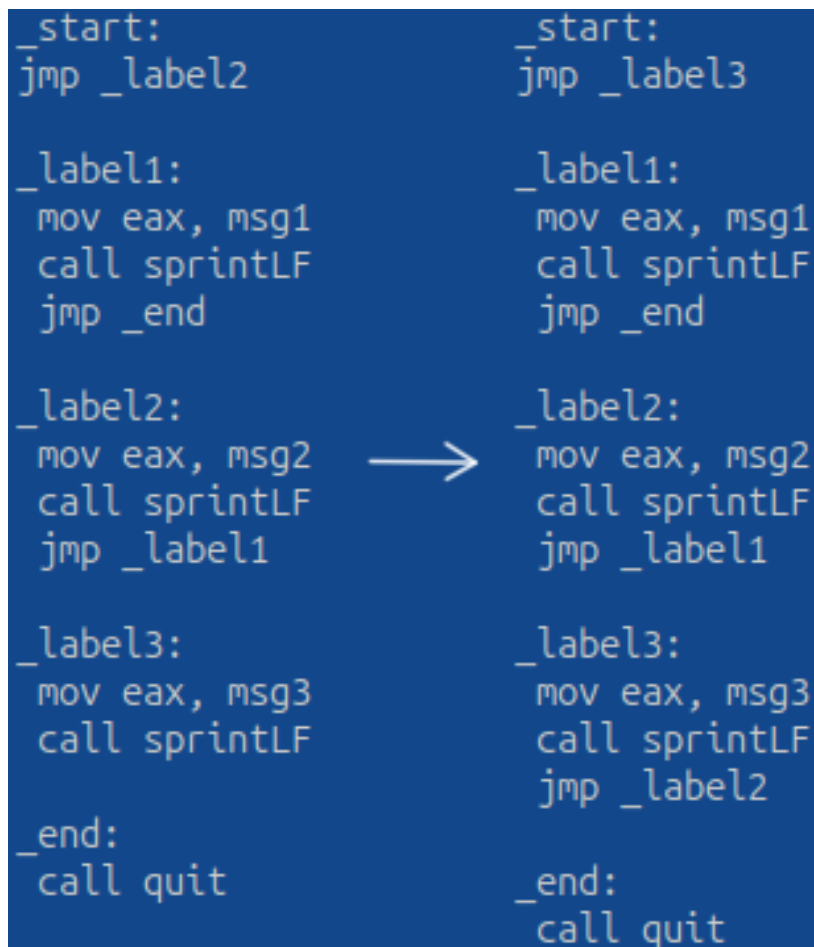
Создадим исполняемый файл.

```
knmuhina@virtualbox:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 1
knmuhina@virtualbox:~/work/arch-pc/lab07$
```

Рисунок 2.4: Изменённая программа lab7-1.

Теперь программа сначала выполняет инструкции, начиная с метки `_label2`, затем переходит к метке `_label1`, после чего переходит к метке завершения программы, пропуская сообщение №3.

Снова изменим текст программы:



<code>_start: jmp _label2</code>		<code>_start: jmp _label3</code>
<code>_label1: mov eax, msg1 call sprintf jmp _end</code>		<code>_label1: mov eax, msg1 call sprintf jmp _end</code>
<code>_label2: mov eax, msg2 call sprintf jmp _label1</code>	→	<code>_label2: mov eax, msg2 call sprintf jmp _label1</code>
<code>_label3: mov eax, msg3 call sprintf</code>		<code>_label3: mov eax, msg3 call sprintf jmp _label2</code>
<code>_end: call quit</code>		<code>_end: call quit</code>

Рисунок 2.5: Изменение программы lab7-1.

Создадим исполняемый файл.

```
knmuhina@virtualbox:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
knmuhina@virtualbox:~/work/arch-pc/lab07$
```

Рисунок 2.6: Изменённая программа lab7-1.

Теперь программа выводит сообщения в обратном порядке.

Рассмотрим одну из инструкций условного перехода на примере следующей программы. Данная программа будет определять и выводить на экран

наибольшую из 3 целочисленных переменных А, В и С. Значения А и С уже заданы в программе, а В будет вводиться с клавиатуры.

Создадим файл lab7-2.asm, введём в файл текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл. Проверим работу программы, задавая разные значения В.

```
knmuhina@virtualbox:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
knmuhina@virtualbox:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 5
Наибольшее число: 9
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 21
Наибольшее число: 21
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 0
Наибольшее число: 9
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./lab7-2
Введите В: 15
Наибольшее число: 15
knmuhina@virtualbox:~/work/arch-pc/lab07$
```

Рисунок 2.7: Программа lab7-2.

Программа работает корректно.

Обратим внимание, что в данной программе А и С сравниваются как символы, а переменная В и максимум из А и С - как числа. Здесь это сделано для демонстрации того, как сравниваются данные. Можно упростить программу и сравнивать все переменные как символы, либо как числа. Последнее позволит корректно проводить арифметические операции над переменными.

2. Изучение структуры файла листинга.

Чтобы получить файл листинга программы, нужно указать ключ -l и задать имя файла листинга в командной строке. Создадим файл листинга программы из файла lab7-2.asm и тут же откроем его при помощи редактора mcedit.

```
knmuhina@virtualbox:~/work/arch-pc/lab07$ nasm -f elf -l lab7-2.lst lab7-2.asm
knmuhina@virtualbox:~/work/arch-pc/lab07$ mcedit lab7-2.lst
```

Рисунок 2.8: Создание файла листинга lab7-2.lst.

Файл листинга выглядит следующим образом:

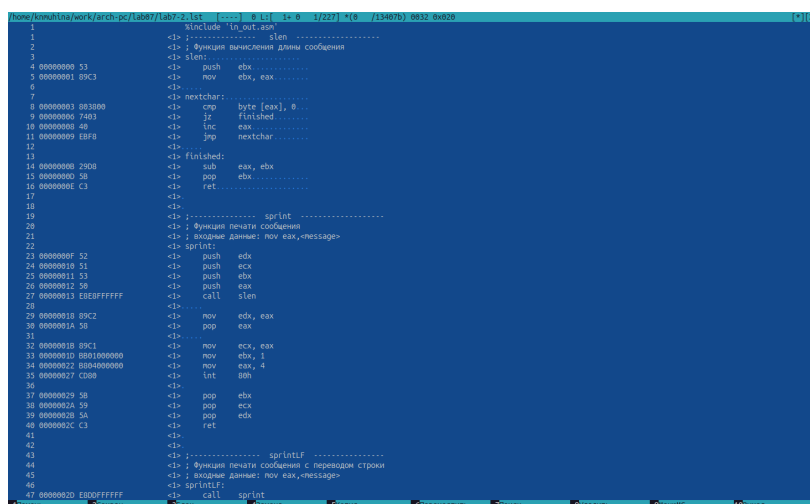


Рисунок 2.9: Файл листинга lab7-2.lst.

Сразу обратим внимание, что в файле листинга номер строки листинга не всегда совпадает с номером строки текста программы. Разберём некоторые строки листинга.

Строка листинга №1 указывает на строку текста программы №1, подключающий к программе lab7-2 подпрограммы из файла in_out.asm и не содержит других данных. Далее в листинге описываются подпрограммы из самого файла.

Разберём строки листинга самого текста программы lab7-2. Строки листинга

```
6 00000035 32300000      A dd '20'
```

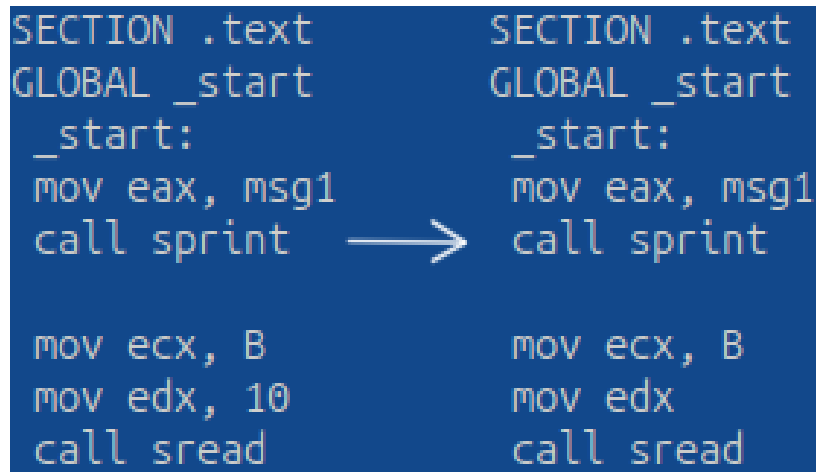
указывают подробную информацию по следующей структуре: номер строки текста программы, адрес (смещение машинного кода от начала сегмента .data), машинный код строки, представленный шестнадцатеричной последовательностью, и сам исходный текст программы - задание переменных A и C.

Строка листинга

37 0000012F E868FFFFFF call atoi

по такой же структуре указывает подробную информацию о инструкции вызова функции atoi.

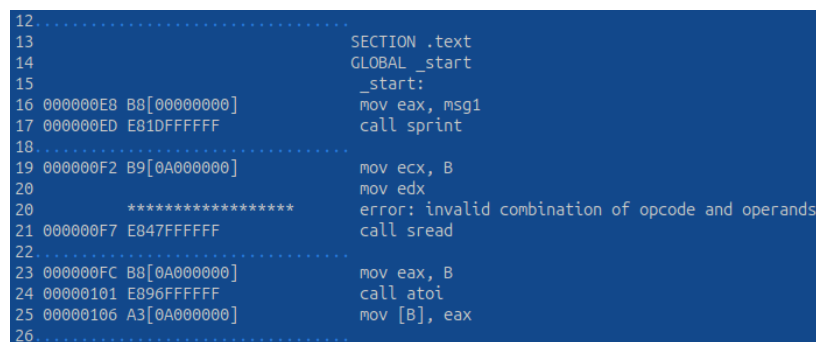
Листинг программы также может содержать информацию о возможных предупреждениях и ошибках, возникающих в ходе выполнения инструкций. Откроем файл с программой и из одной из инструкций удалим операнд.



The diagram illustrates the removal of an operand from an instruction. It shows two versions of assembly code side-by-side, separated by a large right-pointing arrow. The left version shows a sequence of instructions: `SECTION .text`, `GLOBAL _start`, `_start:`, `mov eax, msg1`, `call sprint`, `mov ecx, B`, `mov edx, 10`, and `call sread`. The right version shows the same sequence, but the `call sprint` instruction has been modified to `call` only, with the `sprint` operand removed.

Рисунок 2.10: Изменение программы lab7-2.

Заново создадим листинг программы и откроем его.



The screenshot shows a list of assembly instructions with their corresponding hex addresses and mnemonics. The instructions are: `SECTION .text`, `GLOBAL _start`, `_start:`, `mov eax, msg1`, `call sprint`, `mov ecx, B`, `mov edx`, `error: invalid combination of opcode and operands`, `call sread`, `mov eax, B`, `call atoi`, and `mov [B], eax`. The error message is displayed on line 20, indicating an invalid combination of opcode and operands for the instruction `mov edx`.

Рисунок 2.11: Изменённый фрагмент листинга lab7-2.

К строке текста программы №20 листинг добавил примечание об ошибке в некорректной записи операндов в инструкции.

3 Задания для самостоятельной работы

Создадим файл task1.asm и напишем программу нахождения наименьшей из 3 целочисленных переменных. Значения переменных были выбраны в соответствии с вариантом, полученным при выполнении лабораторной работы №6.

Листинг 1. Программа task1.asm

```
%include 'in_out.asm'

SECTION .data
msg DB 'Наименьшее число: ', 0h
A dd '99'
B dd '29'
C dd '26'

SECTION .bss
min resb 10

SECTION .text
GLOBAL _start
_start:
```

```
mov ecx, [A]
mov [min], ecx
```

```
mov eax, min
call atoi
mov [min], eax
```

```
mov eax, B
call atoi
mov [B], eax
```

```
cmp [B], ecx
jg check_C
mov ecx, [B]
mov [min], ecx
```

```
check_C:
    mov eax, C
    call atoi
    mov [C], eax
```

```
mov ecx, [min]
cmp [C], ecx
jg fin
mov ecx, [C]
mov [min], ecx
```

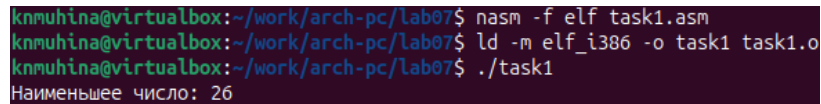
```
fin:
    mov eax, msg
```

```

call sprint
mov eax, [min]
call iprintLF
call quit

```

Создадим исполняемый файл и проверим работу программы.



```

knmuhina@virtualbox:~/work/arch-pc/lab07$ nasm -f elf task1.asm
knmuhina@virtualbox:~/work/arch-pc/lab07$ ld -m elf_i386 -o task1 task1.o
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./task1
Наименьшее число: 26

```

Рисунок 3.1: Программа task1.asm.

Программа работает корректно.

Теперь создадим файл task2.asm и напишем программу вычисления значения заданной функции. Вид функции определён вариантом, полученным при выполнении лабораторной работы №6.

Листинг 2. Программа task2.asm

```

%include 'in_out.asm'

SECTION .data
var DB 'Вычисление функции.', 0h
msg1 DB 'Введите x: ', 0h
msg2 DB 'Введите a: ', 0h
msgR DB 'Результат: ', 0h
C dd 5

SECTION .bss
X resb 10
A resb 10
res resb 40

```

```
SECTION .text
GLOBAL _start
_start:
    mov eax, var
    call sprintf

    mov eax, msg1
    call sprintf

    mov ecx, X
    mov edx, 10
    call sread

    mov eax, X
    call atoi
    mov [X], eax

    mov eax, msg2
    call sprintf

    mov ecx, A
    mov edx, 10
    call sread

    mov eax, A
    call atoi
    mov [A], eax

    mov ecx, [C]
```

```
cmp ecx, [X]
```

```
jg func
```

```
mov eax, [X]
```

```
sub eax, [C]
```

```
mov [res], eax
```

```
jmp _res
```

```
func:
```

```
mov ebx, [X]
```

```
mov eax, [A]
```

```
mul ebx
```

```
mov [res], eax
```

```
_res:
```

```
mov eax, msgR
```

```
call sprint
```

```
mov eax, [res]
```

```
call iprintLF
```

```
call quit
```

Создадим исполняемый файл и проверим работу программы на основе значений из таблицы, представленной в лабораторной работе.

```
knmuhina@virtualbox:~/work/arch-pc/lab07$ nasm -f elf task2.asm
knmuhina@virtualbox:~/work/arch-pc/lab07$ ld -m elf_i386 -o task2 task2.o
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./task2
Вычисление функции.
Введите x: 3
Введите a: 7
Результат: 21
knmuhina@virtualbox:~/work/arch-pc/lab07$ ./task2
Вычисление функции.
Введите x: 6
Введите a: 4
Результат: 1
knmuhina@virtualbox:~/work/arch-pc/lab07$
```

Рисунок 3.2: Программа task2.asm.

Программа работает корректно.

4 Выводы

В ходе выполнения лабораторной работы мы изучили команды условного и безусловного переходов в NASM, приобрели навыки написания программ с использованием переходов и ознакомились с назначением и структурой файла листинга.

Список литературы

1. Файл «Лабораторная работа №7. Команды безусловного и условного переходов в Nasm. Программирование ветвлений.pdf»